

Digital Signal Processing

TUTORIAL 6 NOISE SIGNAL AND FILTERING

PIANO SYNTH AND SIGNAL ANALYZER



BIRMINGHAM CITY
University

1. Learning Outcomes

- Understand random noise and normal distribution.
- be able to use Matlab function `randn()` to generate pseudo random noise.
- be able to use `filterDesigner` to manipulate signal in frequency domain.
- be able to create customised function.

2. Noise Signal

In practice, natural random noise can be a useful signal. We will look how to generate noise. Noise may be an inherent part of the signal being measured, arise from imperfections in the data acquisition system (for example quantisation noise), or be introduced as an unavoidable by-product of some DSP operation. Statistics and probability allow these disruptive features to be measured and classified, the first step in developing strategies to remove the offending components. On the other hand, noise can be directly used as components in modern music.

3. Digital Noise Generation

A common need in DSP is to generate signals that resemble various types of random noise. This is required to test the performance of algorithms that must work in the presence of noise.

Most natural random noises has their **probability distribution function** (PDF) statistically following “**normal distribution**”, also called “**Gaussian distribution**”, which is shown as following equation and figure:

$$P(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

Equation 1

Here μ is mean of noise, σ is standard deviation.

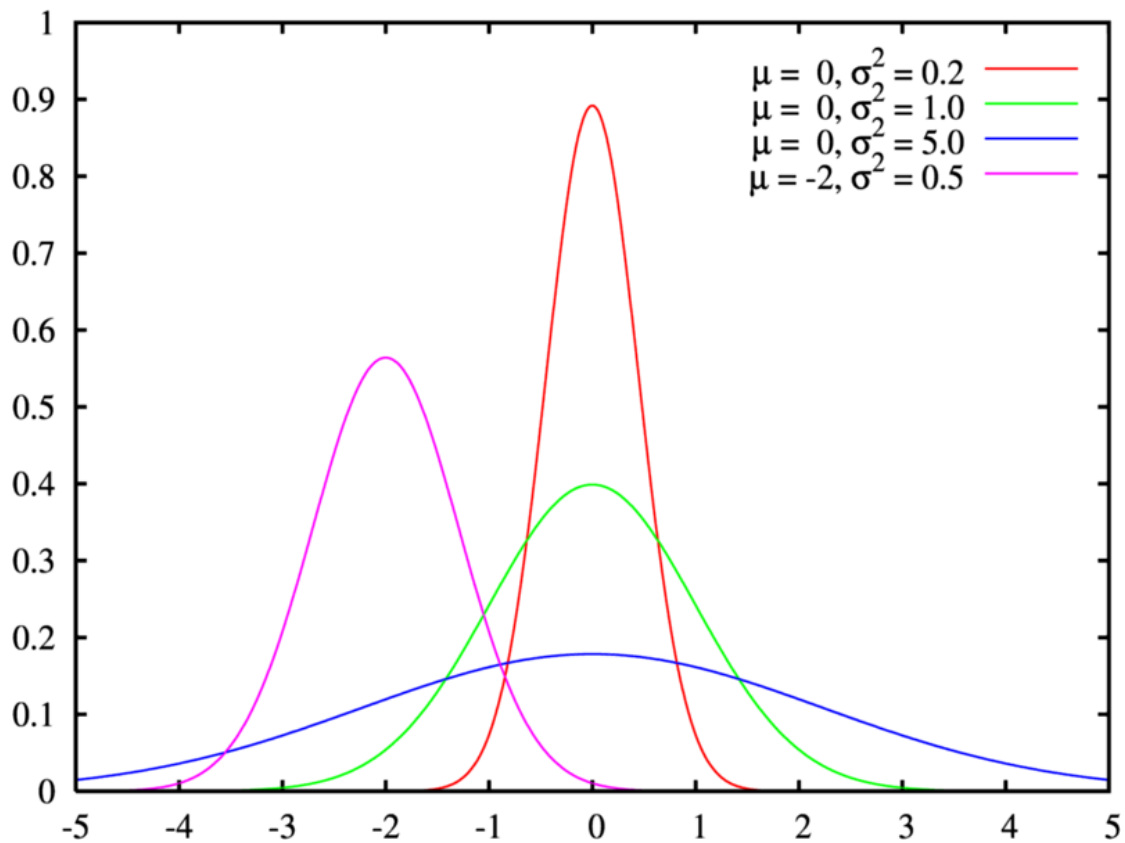


Figure 1 Normal distribution

The properties of normal distribution noise signal will be discussed later. First of all let's create artificial random noise by Matlab.

4. Random number generator

Mathematicians have created algorithms to let software/computers generate pseudo random numbers. In Matlab function "randn()" generates normally distributed random numbers and arrays.

randn

Normally distributed random numbers and arrays

Syntax

```
Y = randn
Y = randn(n)
Y = randn(m,n)
Y = randn([m n])
Y = randn(m,n,p,...)
Y = randn([m n p...])
Y = randn(size(A))
```

Description

`Y = randn` returns a pseudorandom, scalar value drawn from a normal distribution with mean 0 and standard deviation 1.

`Y = randn(n)` returns an n-by-n matrix of values derived as described above.

`Y = randn(m,n)` or `Y = randn([m n])` returns an m-by-n matrix of the same.

`Y = randn(m,n,p,...)` or `Y = randn([m n p...])` generates an m-by-n-by-p-by-... array of the same.

`Y = randn(size(A))` returns an array that is the same size as A.

Matlab by default use “Mersenne twister” algorithm to generate the pseudo random numbers. For a full description of the Mersenne twister algorithm, see <http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html>.

Then we generate 6 randn numbers and store them in array noise:

```
noise = randn(6, 1);
```

In order to simulate background noise in audio signal, we need to determine some parameters

5. Exercise

Create bandlimited noise

- a. Generate 2 second noise with sampling frequency as 22050 Hz in Matlab:

```
NoiseFs = 22050;
NoiseDur = 2;
Noise = ?????; % Replace question mark with your code
sound(Noise,NoiseFs);
```

Table 1

- b. Import signal into signalAnalyzer

Activate SPTool by typing

```
signalAnalyzer
```

The signalAnalyzer window is displayed.

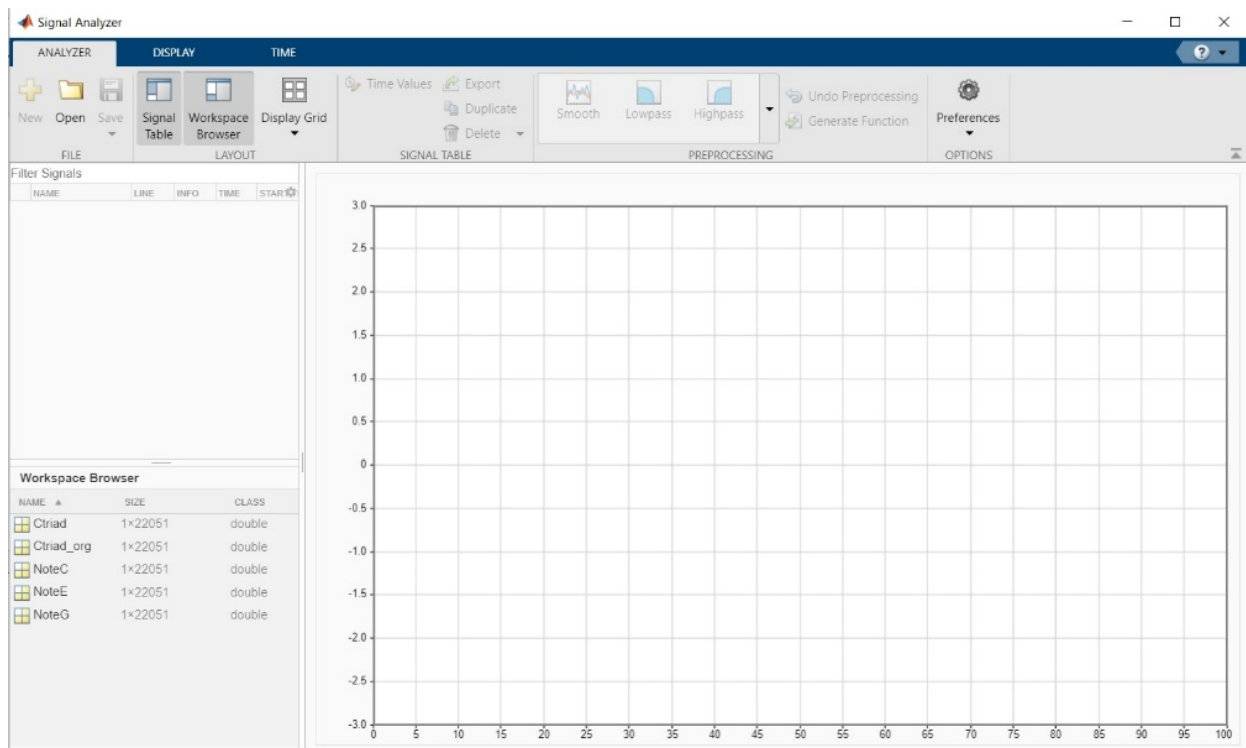


Figure 2 signalAnalyzer window

Import noise signal by dragging the Noise variable from the workspace browser into the plotting window. The signal should appear in the time domain plot area of the window. Next, we must ensure the signal is correct by selecting the sample rate and start time. In the filter signals section, right click on the variable you have imported, and select "Time Values", set the "Time specification" option to "Set sample rate and start time", and then input the sample frequency, and set start time to "0" seconds and hit ok. The x axis of the plot should covert from samples, to time (seconds).

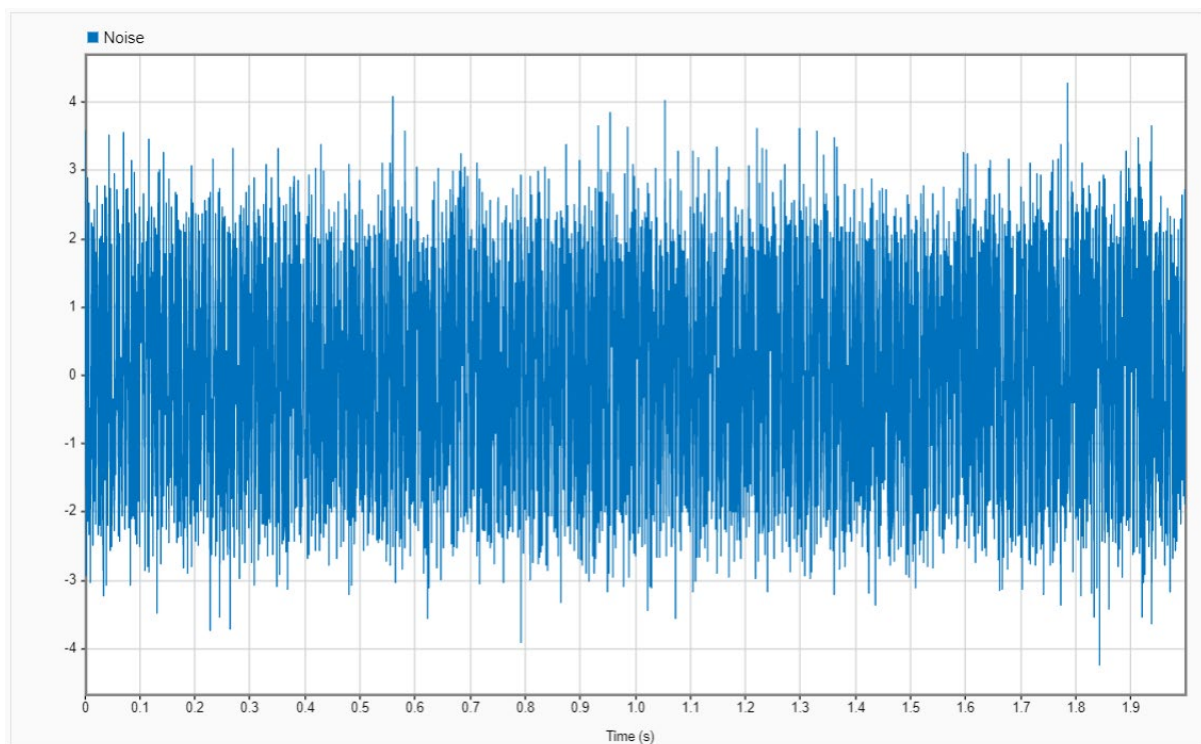


Figure 3 Noise signal

c. Create Filter

Now we will use a tool called `filterDesigner` to create a filter.

Into the command window type:

```
filterDesigner
```

You will be presented with the following window:

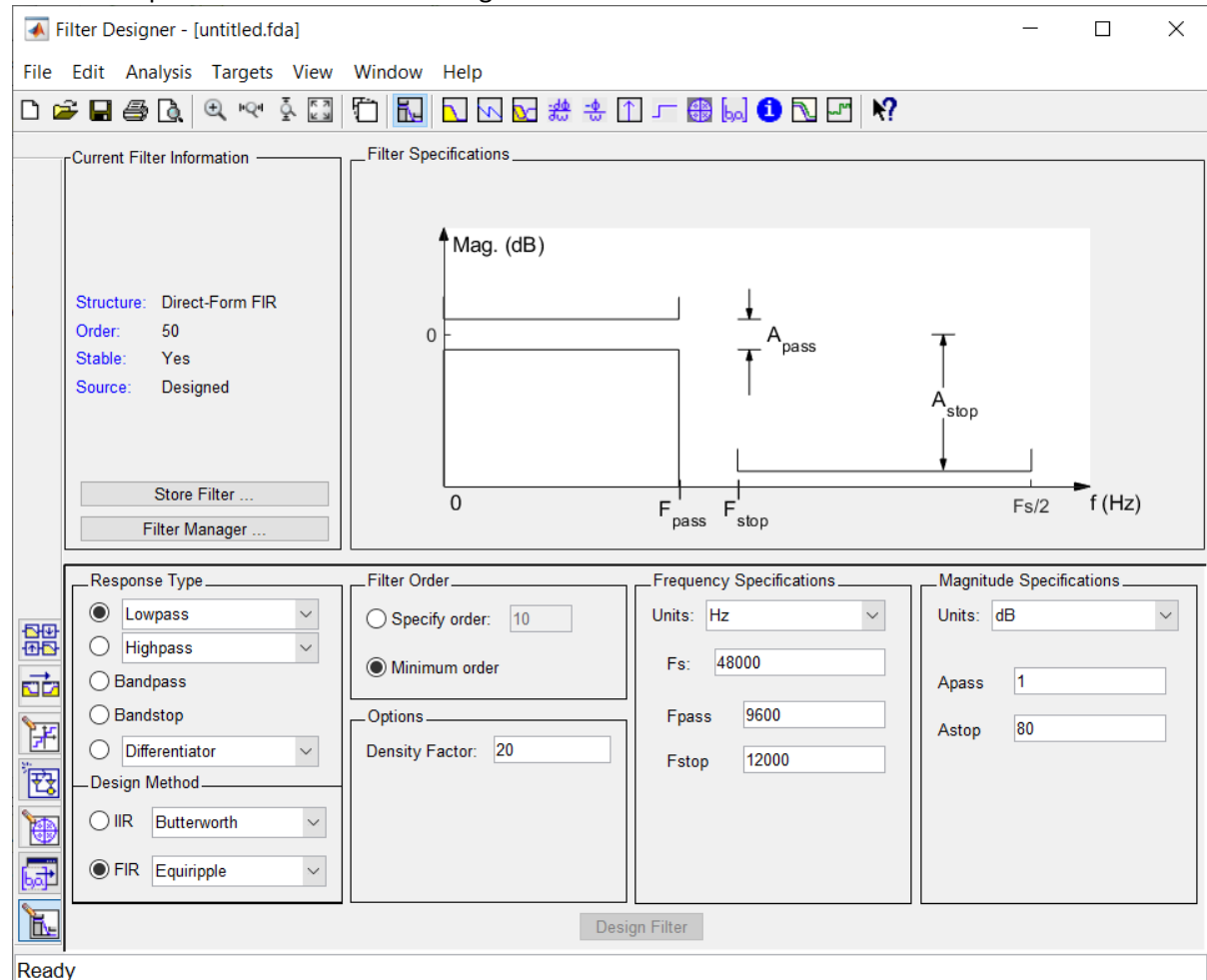


Figure 4: filterDesigner window

Let's look at some of the parameters. First, we have the response type, to choose what kind of filter response we need select a response from the "Response Type" section in the lower left corner of the screen. We shall start with a bandpass filter. Select **Bandpass**.

Next, we should select the method. We have a choice from Infinite impulse response (IIR) filters and Finite impulse response (FIR) filters. We will learn more about these different types in future labs. For now, select **FIR** and from the drop-down menu select **Equiripple**.

Keep the filter order set to minimum order, the filter designer software will determine the minimum order filter for us based upon our other filter parameters. Manually specifying the filter order allows for more efficient filters, however this may come at the expense of our desired filter response. Leave Density factor set at its default value of 20.

Enter the correct sample frequency of your noise signal and input the following parameter values, then select design filter.

Fstop1	3000 Hz
Fpass1	4000 Hz
Fpass2	5500 Hz
Fstop2	6500 Hz
Astop1	75 dB
Apass	1
Astop2	75 dB

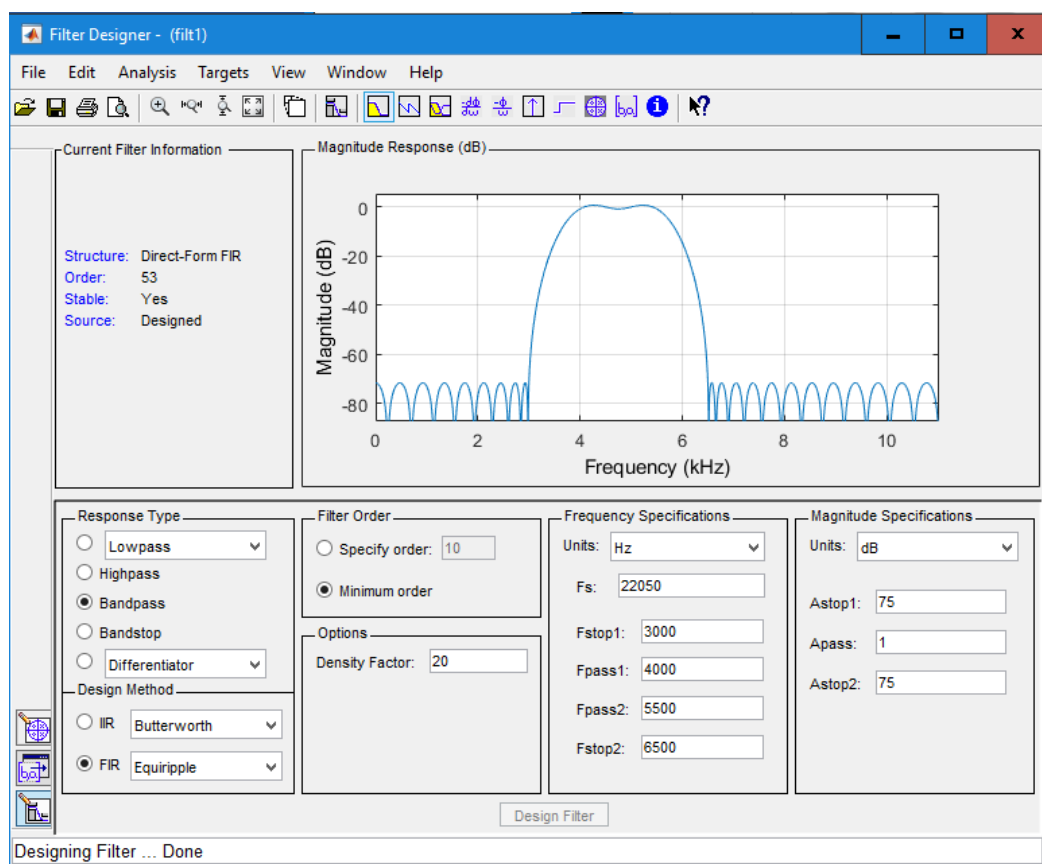


Figure 5: Filter Design

d. Apply Filter into the noise signal

Now we need to export the filter from the filterDesigner app back into the matlab workspace, there are a number of different ways this can be done. Select *file* then *export* (Or Ctrl + E, cmd +E).

Select export to **workspace**. Export as **object**. Give your filter a name, then press export. There should be a new filter object variable in your workspace. This object contains the impulse response of the filter we designed. We can then use convolution to apply the impulse response to filter the signal.

- 1). Investigate the filter. Filter object has many different subfields. The most important subfields are "**filt1.Numerator**" and "**filt1.Denominator**", which correspond to the filter's transfer function (tf)'s numerators and denominators. For FIR filter, the value of "**filt1.Denominator**" is always equal to "1".
- 2). Apply exported filter using both convolution and filtering methods. Search Matlab help file to find how to use filter function as "**y = filter(b,a,X)**". complete the following code to filter the noise using both convolution and filter functions:

```

a = 1;

b = filt1.numerator;

stem(b); %This plots the impulse response of your filter

newnoise1 = conv(????);

newnoise2 = filter(??????);

%% playback them

sound(newnoise1, NoiseFs);

sound(newnoise2, NoiseFs);

len = ?????

```

Complete above code to find out the different lengths between two filtered noise "newnoise1" and "newnoise2"

- 3). Use signal analyser to plot the spectrum of your noise signal and your filtered signals

Independent Task 1

Instead of using MATLAB's built-in `filter()` and `conv()` functions to apply our filter, modify the following code for applying convolution such that it runs using any size impulse response and turn this into your own convolution function.

```
Signal = [1,2,3,4];
IR = [1, 1, 1];
Output = zeros(1,6)
for i = 1 : 4
    for ii = 1 : 3
        Output(i+ii-1) = Output(i+ii-1)+ signal(i).*IR(ii);
    end
end
```

2) Create a low pass filter using `filterDesigner`, and use your own convolution function to apply it to a signal.

Independent Task 2

Research and investigate the theoretical method to create digital **pink noise**. Describe the algorithm behind them and the steps of implementation in Matlab. If you can implement them in Matlab, comment your code in detail. See

(https://ccrma.stanford.edu/~jos/sasp/Example_Synthesis_1_F_Noise.html) for examples.

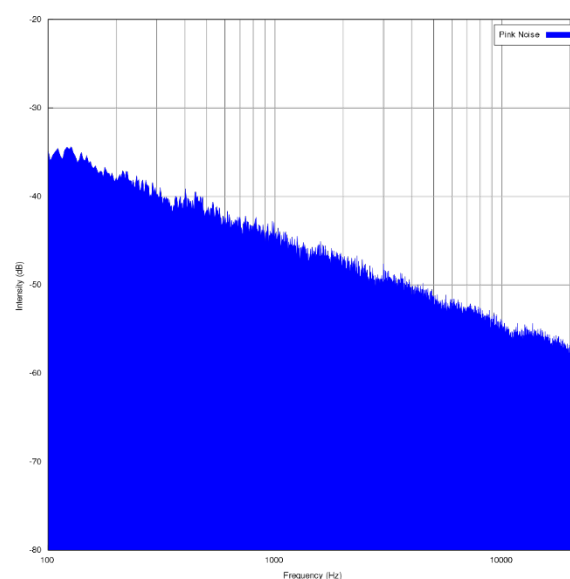


Figure 4 Pink Noise Spectrum